LAPORAN PRAKTIKUM

TEKNOLOGI IOT II

Modul 4: Komunikasi Asinkron dan WebSockets

Disusun oleh :

Nama : syafa sartika putri aryanti

NPM : 25782056

Kelas : TRI-3B

Dosen:

Agus Ambarwari, S.Pd., M.Kom.

Dr. Ir. Septafiansyah Dwi Putra

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026/2027

4.6 Latihan

- Latihan 1 (Teori Jaringan): Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh HTTP* HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

  JAWAB: WebSockets jauh lebih hemat *bandwidth* dan tidak membuat layar berkedip karena menggunakan **koneksi permanen dua arah (*persistent full-duplex connection*)** melalui satu saluran tunggal. Ketika ada data baru, WebSocket hanya mengirimkan paket data berukuran sangat kecil (seperti teks JSON) dan langsung memperbarui elemen HTML tertentu di layar menggunakan JavaScript (*DOM manipulation*), tanpa mengganggu bagian halaman lainnya.

  Sebaliknya, metode *Meta Refresh* HTTP memaksa *browser* untuk memuat ulang (*reload*) **seluruh halaman web** dari awal secara berkala. Hal ini membuat *browser* harus mendownload ulang seluruh struktur HTML, CSS, dan *header* HTTP yang besar secara terus-menerus (sehingga boros kuota/data), serta menyebabkan layar berkedip (*flicker*) karena halaman harus dirender ulang dari kosong setiap kali jeda *refresh* tercapai.

- Latihan 2 (Analisis Waktu/Timer): Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan NodeMCU) jika Anda bersikeras nekat memanggil sintaks delay(3000) di dalam siklus fungsi void loop() pada draf arsitektur kompleks ini?

  JAWAB: Pemanggilan fungsi delay(3000) di dalam siklus loop() pada arsitektur WebSocket akan menyebabkan **blokade total pada CPU mikrokontroler (pemblokiran non-asinkron)**. Selama jeda 3 detik tersebut, ESP8266 akan berhenti total dari segala aktivitas — tidak dapat mendeteksi tekanan *push button* fisik, mengabaikan pembacaan sensor, dan gagal memproses atau merespons lalu lintas data masuk/keluar dari klien *WebSocket*. Akibat fatalnya, koneksi *WebSocket* akan mengalami *timeout* dan terputus secara sepihak (*dropped connection*), serta sistem kendali *real-time* dua arah antara tombol fisik dan antarmuka web menjadi tidak responsif atau macet total.

- **Latihan 3 (Modifikasi Front-End):** Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML () apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?

  JAWAB: Berdasarkan arsitektur *front-end dashboard* web pada modul ini, elemen HTML dengan tag ID () yang menjadi target utama injeksi atau pengubahan teks secara dinamis oleh skrip JavaScript meliputi:

1. **temperature** (atau **suhu**) — Berfungsi sebagai target penampung untuk memperbarui nilai angka suhu hasil pembacaan sensor DHT11 secara *real-time* di layar.
2. **ledState** (atau **ledStatus**) — Berfungsi sebagai target untuk mengubah teks status indikator lampu LED (misalnya berganti antara teks "ON" atau "OFF") secara instan ketika terjadi perubahan status baik dari penekanan tombol fisik maupun dari klik tombol di web.

- **Latihan 4 (Pengembangan Telemetri Sensor - Kelembapan):** Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via WebSockets:

    i. **Sisi Back-End (C++):** Buat variabel penampung (misal: String currentHum = "--";). Baca nilai kelembapan menggunakan fungsi dht.readHumidity() di dalam siklus timer *non-blocking* millis(). Perbarui fungsi notifyClients() agar menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke *browser* (contoh format: {"led":"...", "suhu":"...", "hum":"..."}).

    ii. **Sisi Front-End (HTML & JavaScript):** Tambahkan elemen kartu ( ) baru pada kerangka HTML sebagai penampil kelembapan (misal: <h2>Kelembapan: -- %</h2>). Kemudian, modifikasi fungsi JavaScript onMessage(event) agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen humValue. Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!

  Jawab:

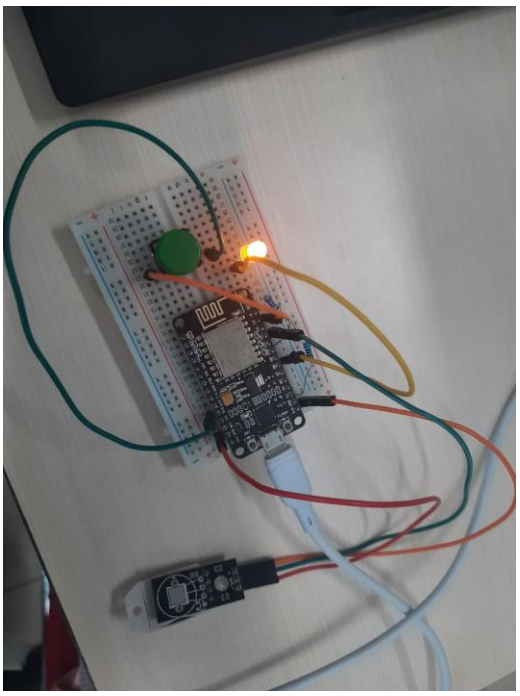
  Penjelasannya :

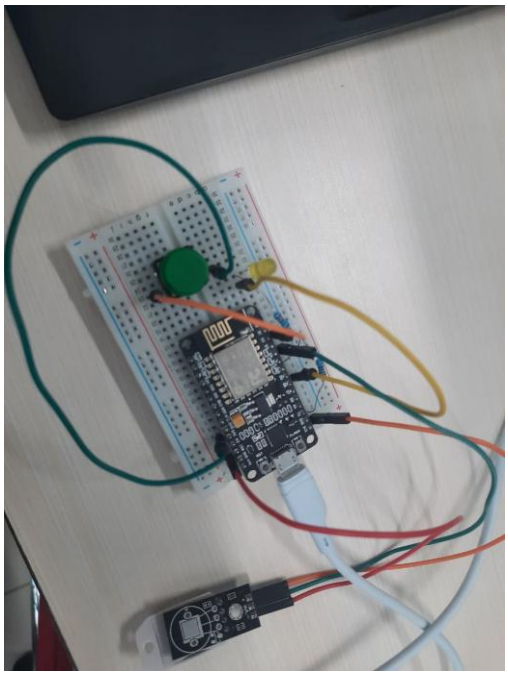
  Telemetri Suhu: Menampilkan data pembacaan suhu dari sensor DHT11 secara kontinu.

  Telemetri Kelembapan (*Fitur Baru*): Berhasil memuat dan menampilkan persentase kelembapan udara (hum) yang dikirimkan oleh NodeMCU melalui paket data JSON, lalu disuntikkan secara dinamis ke elemen .

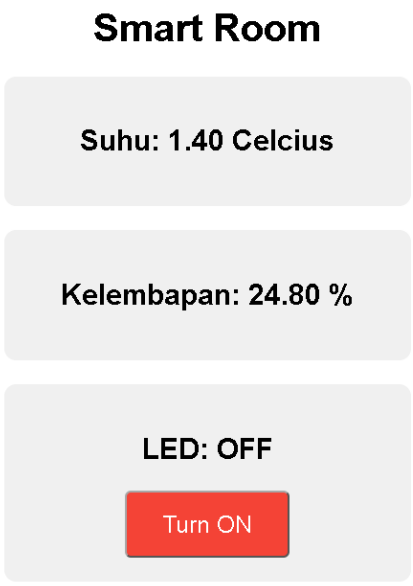
  Kendali & Status LED: Menampilkan status visual lampu secara *real-time* (ON/OFF) yang tersinkronisasi dua arah antara penekanan *push button* fisik di *breadboard* dan klik tombol interaktif di antarmuka web.

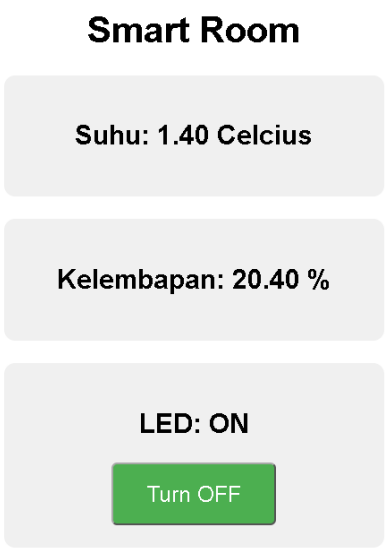
HASILNYAA :





Hasil web nya:





Code modifikasi :

Blink.ino

```
#include <ESP8266WiFi.h>
#include <ESPAsyncTCP.h>
#include <ESPAsyncWebServer.h>
#include <DHT.h>

const char* ssid = "syafa";
const char* password = "apayaaaaa";

// Konfigurasi Pin
const byte dhtPin = 2;        // D4 (GPIO 2)
const byte buttonPin = 4;     // D2 (GPIO 4)
const byte ledPin = 12;       // D6 (GPIO 12)

DHT dht(dhtPin, DHT11);

// Variabel Pelacak Status (State & Cache)
bool ledState = false;
String currentTemp = "--";
String currentHum = "--";     // VARIABEL BARU: Penampung data kelembapan
int buttonState;
int lastButtonState = LOW;
unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 50;
unsigned long lastTime = 0;

// Inisialisasi Async Web Server (port 80) & WebSocket (rute /ws)
AsyncWebServer server(80);
AsyncWebSocket ws("/ws");

// ---------------- HTML & JAVASCRIPT (FRONT-END) ----------------
const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Real-Time IoT Web</title>
  <style>
    body { font-family: Arial; text-align: center; }
    .card { background: #f0f0f0; margin: 20px auto; padding: 20px; max-width: 300px; border-radius: 10px; }
    button { padding: 15px 30px; font-size: 20px; border-radius: 5px; cursor: pointer; color: white;}
    .btn-on { background-color: #4CAF50; }
    .btn-off { background-color: #f44336; }
  </style>
</head>
<body>
  <h1>Smart Room</h1>
  <div class="card">
    <h2>Suhu: <span id="tempValue">--</span> Celcius</h2>
  </div>
```

Blink.ino

```
  <!-- KARTU BARU: Elemen HTML untuk Menampilkan Kelembapan -->
  <div class="card">
    <h2>Kelembapan: <span id="humValue">--</span> %</h2>
  </div>

  <div class="card">
    <h2>LED: <span id="ledStatus">OFF</span></h2>
    <button id="toggleBtn" class="btn-off" onclick="toggleLed()">Turn ON</button>
  </div>

  <script>
    // Membuka terowongan koneksi WebSocket ke alamat IP ESP8266
    var gateway = `ws://${window.location.hostname}/ws`;
    var websocket;

    // Menjalankan inisiasi saat halaman pertama kali dimuat
    window.addEventListener('load', onLoad);
    function onLoad(event) { initWebSocket(); }

    function initWebSocket() {
      websocket = new WebSocket(gateway);
      websocket.onopen    = onOpen;
      websocket.onclose   = onClose;
      websocket.onmessage = onMessage;
    }

    function onOpen(event) { console.log('WebSocket Terkoneksi'); }
    function onClose(event) { setTimeout(initWebSocket, 2000); }

    // Dipanggil saat tombol di layar web ditekan
    function toggleLed(){
      websocket.send('toggle');
    }

    // Menangkap paket data JSON yang "didorong" (PUSH) oleh C++ NodeMCU
    function onMessage(event) {
      var dataObj = JSON.parse(event.data);

      // Menyuntikkan teks angka Suhu ke dalam id "tempValue" HTML
      if(dataObj.suhu !== undefined) {
        document.getElementById('tempValue').innerHTML = dataObj.suhu;
      }

      // MODIFIKASI BARU: Menyuntikkan teks angka Kelembapan ke id "humValue" HTML
      if(dataObj.hum !== undefined) {
        document.getElementById('humValue').innerHTML = dataObj.hum;
      }

      // Merombak UI Tombol & Teks sesuai status hardware
      if(dataObj.led !== undefined) {
```

Blink.ino

```
      // Merombak UI Tombol & Teks sesuai status hardware
      if(dataObj.led !== undefined) {
        var btn = document.getElementById('toggleBtn');
        var status = document.getElementById('ledStatus');
        if(dataObj.led == "1"){
          status.innerHTML = "ON";
          btn.innerHTML = "Turn OFF";
          btn.className = "btn-on";
        } else {
          status.innerHTML = "OFF";
          btn.innerHTML = "Turn ON";
          btn.className = "btn-off";
        }
      }
    }
  </script>
</body>
</html>
)rawliteral";

// ---------------- BACK-END & WEBSOCKET LOGIC ----------------

// MODIFIKASI: Menyisipkan data "hum" ke dalam paket JSON
void notifyClients() {
  String jsonString = "{\"led\":\"" + String(ledState ? 1 : 0) + "\", ";
  jsonString += "\"suhu\":\"" + currentTemp + "\", ";
  jsonString += "\"hum\":\"" + currentHum + "\"}"; // Menyertakan kelembapan
  ws.textAll(jsonString);
}

// Handler pesan masuk (dari klik browser)
void handleWebSocketMessage(void *arg, uint8_t *data, size_t len) {
  AwsFrameInfo *info = (AwsFrameInfo*)arg;
  if (info->final && info->index == 0 && info->len == len && info->opcode == WS_TEXT) {
    data[len] = 0;
    if (strcmp((char*)data, "toggle") == 0) {
      ledState = !ledState;
      notifyClients();
    }
  }
}

// Event handler WebSocket bawaan AsyncWebServer
void onEvent(AsyncWebSocket *server, AsyncWebSocketClient *client, AwsEventType type,
             void *arg, uint8_t *data, size_t len) {
  switch (type) {
    case WS_EVT_CONNECT:
      Serial.printf("Client WebSocket #%u terhubung\n", client->id());
      notifyClients(); // Kirim status terkini kepada klien yang baru buka web
      break;
```

Blink.ino

```
    case WS_EVT_DISCONNECT:
      Serial.printf("Client WebSocket #%u terputus\n", client->id());
      break;
    case WS_EVT_DATA:
      handleWebSocketMessage(arg, data, len);
      break;
  }
}

void setup() {
  Serial.begin(115200);

  // Mengatur pin tombol sebagai input (dengan resistor pull-down eksternal)
  pinMode(buttonPin, INPUT);

  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);
  dht.begin();

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.println("\nIP Address: " + WiFi.localIP().toString());

  server.on("/", HTTP_GET, [](AsyncWebServerRequest *request){
    request->send_P(200, "text/html", index_html);
  });

  // Ikat handler WebSockets ke server utama
  ws.onEvent(onEvent);
  server.addHandler(&ws);
  server.begin();
}

void loop() {
  ws.cleanupClients();

  // 1. Eksekusi perangkat keras LED
  digitalWrite(ledPin, ledState ? HIGH : LOW);

  // 2. Baca Tombol Fisik (Debounce) tanpa nge-freeze program
  int reading = digitalRead(buttonPin);
  if (reading != lastButtonState) {
    lastDebounceTime = millis();
  }
  if ((millis() - lastDebounceTime) > debounceDelay) {
    if (reading != buttonState) {
      buttonState = reading;

      // Karena menggunakan resistor PULL-DOWN eksternal, tombol ditekan = HIGH
      if (buttonState == HIGH) {
        ledState = !ledState;
        notifyClients(); // PUSH DATA KE BROWSER SEKETIKA
      }
    }
  }
  lastButtonState = reading;

  // 3. Baca DHT (Suhu & Kelembapan) setiap 3 detik secara non-blocking
  if ((millis() - lastTime) > 3000) {
    float h = dht.readHumidity();     // Baca Kelembapan
    float t = dht.readTemperature();  // Baca Suhu

    bool dataUpdated = false;

    if(!isnan(h)) {
      currentHum = String(h);
      dataUpdated = true;
    }
    if(!isnan(t)) {
      currentTemp = String(t);
      dataUpdated = true;
    }

    // Jika ada data baru yang valid, push pembaruan ke browser secara otomatis
    if(dataUpdated) {
      notifyClients();
    }

    lastTime = millis();
  }
}
```



Blink.ino

```
#include <ESP8266WiFi.h>
#include <ESPAsyncTCP.h>
#include <ESPAsyncWebServer.h>
#include <DHT.h>

const char* ssid = "syafa";
const char* password = "apayaaaaa";

// Konfigurasi Pin
const byte dhtPin = 2;        // D4 (GPIO 2)
const byte buttonPin = 4;     // D2 (GPIO 4)
const byte ledPin = 12;       // D6 (GPIO 12)

DHT dht(dhtPin, DHT11);

// Variabel Pelacak Status (State & Cache)
bool ledState = false;
String currentTemp = "--";
String currentHum = "--";     // VARIABEL BARU: Penampung data kelembapan
int buttonState;
int lastButtonState = LOW;
```

Output Serial Monitor

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

```
Client WebSocket #3 terhubung
Client WebSocket #3 terputus
Client WebSocket #4 terhubung
Client WebSocket #4 terputus
Client WebSocket #5 terhubung
Client WebSocket #5 terputus
Client WebSocket #6 terhubung
.........
IP Address: 192.168.86.109
Client WebSocket #1 terhubung
```